

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	C.Sandeep	Reg. No.:	192472368
Section / Batch:	AI / 2024	Date:	

Code of Conduct

I C.Sandeep (192472368) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: C.Sandeep

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.	Q1
Inflectional Morphology and Derivational Morphology	Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.	Q2
Finite-State Morphological Parsing	Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.	Q3
Finite-State Morphological Parsing	Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.	Q4
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a Porter Stemmer implementation for words such as <i>connected</i> , <i>connection</i> , <i>connecting</i> , and <i>connectivity</i> ; identify stemming-rule errors and correct the implementation.	Q5
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a morphological normalization program that processes <i>studies</i> , <i>studied</i> , <i>studying</i> , and <i>study</i> ; identify incorrect stemming outputs and modify the code to produce consistent stems.	Q6
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.	Q7

Annexure A – Error Analysis

Question 1 – Error Analysis of Derivational Morphology in Biomedical Text

A biomedical information-retrieval system performs morphological decomposition of words before indexing research articles. The system incorrectly analyzes the words:

1. treatment
2. treatable
3. retreatment
4. treated
5. untreated

The system sometimes removes prefixes or suffixes that carry important morphological information, resulting in incorrect search matches.

Use the PubMed 20k RCT Dataset or another publicly available biomedical corpus for experimentation.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Decompose the selected words into prefixes, roots, and suffixes.
3. Determine whether each identified affix is inflectional or derivational.
4. Explain the root cause of the incorrect morphological analysis.
5. Propose a corrected morphological-analysis strategy.
6. Compare the original and corrected analyses.
7. Explain how the correction could improve biomedical information retrieval.

Answer 1: The words that may produce incorrect analyses are treatment, treatable, retreatment, treated, and untreated. A common error is to remove an affix without checking whether it changes the meaning or grammatical function of the word.

Morphological decomposition: • treatment = treat + -ment. “Treat” is the root/base and -ment is a derivational suffix that forms a noun.

• treatable = treat + -able. “Treat” is the base and -able is a derivational suffix meaning capable of being treated.

• retreatment = re- + treat + -ment. re- is a derivational prefix and -ment is a derivational suffix.

• treated = treat + -ed. -ed is an inflectional suffix expressing past tense/past participle.

• untreated = un- + treat + -ed. un- is a derivational prefix and -ed is an inflectional suffix.

Affix classification: The affixes -ment, -able, re-, and un- are derivational because they create a new lexical meaning or word formation. The suffix -ed in treated and untreated is inflectional because it marks tense/participial form without creating a new lexical category.

Root cause of error: The main cause is treating every prefix or suffix as removable text. A morphological analyzer must distinguish derivational and inflectional affixes and preserve meaningful biomedical terms. For example, removing un- from untreated changes the meaning from “not treated” to “treated.”

Corrected strategy: Use a layered morphological analyzer. First identify possible prefixes, then the lexical root, and finally suffixes. Validate each analysis against a biomedical lexicon and grammatical rules. The analyzer should retain feature information such as NEGATION for un-, REPEAT for re-, DERIVATIONAL_NOUN for -ment, and PAST/PARTICIPLE for -ed.

Original vs corrected analysis: Original analysis may incorrectly reduce retreatment to treat or untreated to treat. Correct analysis keeps the structure: re- + treat + -ment and un- + treat + -ed. This preserves both lexical and grammatical information.

Effect on information retrieval: The corrected analysis improves retrieval because semantically important forms are not collapsed incorrectly. A query about “untreated patients” should not automatically become a query for “treated patients.” At the same time, useful related forms such as treat and treatment can be connected through their common root. This reduces false matches and improves precision and recall.

Question 2 – Error Analysis of Finite-State Morphological Parsing

A social-media NLP system uses a finite-state morphological parser to analyze user-generated text.

The parser produces incorrect analyses for words such as:

1. replayed
2. unhappier
3. disconnected
4. players
5. restarting
6. unreadable

The parser can recognize a single affix correctly but fails when multiple prefixes and suffixes occur in the same word.

Use the Sentiment140 Dataset or another publicly available social-media dataset.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Identify the incorrect FSA/FST transitions responsible for the errors.
3. Modify the finite-state transitions to correctly recognize multiple affixes.
4. Execute the corrected parser on the selected dataset.
5. Compare the parser output before and after correction.
6. Calculate the parsing accuracy before and after correction.
7. Discuss the limitations and computational complexity of the modified finite-state parser.

Answer: The incorrect analyses are mainly caused by a parser that can handle only one affix transition at a time. The required words contain combinations of prefixes, roots, and suffixes.

Incorrect analyses and transitions: • replayed → re- + play + -ed

• unhappier → un- + happy + -er (with y → i spelling change)

• disconnected → dis- + connect + -ed

• players → play + -er + -s

• restarting → re- + start + -ing

• unreadable → un- + read + -able

The faulty FSA/FST design typically returns to a final state after recognizing one affix instead of allowing another valid affix transition.

Corrected transitions: Use a layered finite-state design such as START → PREFIX* → ROOT → DERIVATIONAL_SUFFIX* → INFLECTIONAL_SUFFIX* → FINAL. Prefix states should accept re-, un-, and dis-. Suffix states should accept -er, -able, -ed, -ing, and plural -s. Spelling rules such as happy → happier should be handled by a separate finite-state rewrite rule.

Example corrected output: replayed → re + play + ed [PAST]

unhappier → un + happy + er [COMPARATIVE]

disconnected → dis + connect + ed [PAST]

players → play + er + s [AGENT, PLURAL]

restarting → re + start + ing [PROGRESSIVE]

unreadable → un + read + able [NEGATION, DERIVATION]

Accuracy calculation: If a test set contains 100 manually checked words and the original parser correctly analyzes 72, $\text{accuracy} = 72/100 \times 100 = 72\%$. If the corrected parser correctly analyzes 91 words, $\text{accuracy} = 91/100 \times 100 = 91\%$. Therefore, the improvement is 19 percentage points. These figures are an evaluation example; an actual dataset run should use its manually verified gold annotations.

Limitations and complexity: Finite-state parsing is efficient and deterministic for regular morphological patterns, but it may struggle with irregular forms, ambiguous analyses, spelling changes, and productive word formation. For a word of length n , a well-designed finite-state parser can operate approximately in $O(n)$ time when the number of states and transitions is fixed. Multiple analyses can increase the search cost if the system keeps alternative paths.

Question 3 – Identify the Error, Rectify It, and Analyze the Output

The following program is intended to perform stemming on a collection of documents.

```
from nltk.stem import PorterStemmer
import pandas as pd
ps = PorterStemmer()
data = pd.read_csv("news.csv")
data["Processed"] = data["Text"].apply(ps.stem)
print(data["Processed"].head())
```

The program produces unexpected results and does not correctly process complete sentences.

Tasks

1. Identify the programming and preprocessing errors.
2. Explain why the output is incorrect.
3. Correct the program so that tokenization and stemming are performed appropriately.
4. Execute the corrected program using the AG News Dataset or another publicly available news corpus.
5. Compare:
 - Original text
 - Tokens
 - Stemmed tokens
6. Identify at least 20 problematic stemming cases and classify them as inflectional or derivational.
7. Explain how the corrected program improves morphological preprocessing.

1. Programming and preprocessing errors: The main error is applying `PorterStemmer.stem()` directly to an entire sentence. `PorterStemmer` expects one word at a time, not a complete sentence. The program also does not tokenize text, remove unwanted punctuation, or preserve a clean token sequence.

2. Why the output is incorrect: A complete sentence passed to `ps.stem()` is treated as one string, so the stemmer cannot apply word-level suffix rules correctly. For example, “The players are running” should be processed as separate tokens, not as one sentence string.

3. Corrected program: ``python

```
import pandas as pd
import re
from nltk.stem import PorterStemmer
```

```
ps = PorterStemmer()
data = pd.read_csv("news.csv")
```

```
def preprocess(text):
    tokens = re.findall(r"[A-Za-z]+", str(text).lower())
    stems = [ps.stem(token) for token in tokens]
    return tokens, stems
```

```
data[["Tokens", "Stemmed_Tokens"]] = data["Text"].apply(
    lambda x: pd.Series(preprocess(x))
)
```

```
print(data[["Text", "Tokens", "Stemmed_Tokens"]].head())
````
```

4–5. Example execution and comparison: Example sentence: “The players are connecting devices.”

Tokens: the, players, are, connecting, devices

Stemmed tokens: the, player, are, connect, devic

The original program incorrectly attempts to stem the complete sentence. The corrected program performs tokenization first and then stems each token.

6. Twenty problematic stemming cases: 1. studies → studi (inflectional)
2. studied → studi (inflectional)
3. studying → studi (inflectional)
4. players → player (inflectional)
5. boxes → box (inflectional)
6. cars → car (inflectional)
7. connected → connect (inflectional)
8. connecting → connect (inflectional)
9. organized → organ (inflectional/over-stemming risk)
10. cats → cat (inflectional)
11. dogs → dog (inflectional)
12. happiness → happi (derivational)
13. national → nation (derivational-related reduction)
14. nationality → nation (derivational)
15. nationalize → nation (derivational)
16. readable → readabl (derivational)
17. kindness → kind (derivational)
18. organization → organ (derivational/over-stemming risk)
19. useful → us (derivational/over-stemming risk)
20. relational → relat (derivational/over-stemming risk)

Porter stemming is rule-based, so some outputs are not valid English words. This is a limitation of stemming rather than lemmatization.

7. Improvement: The corrected pipeline separates tokenization from stemming, produces consistent word-level stems, reduces vocabulary variation, and makes the output more suitable for indexing and machine-learning features. It also makes errors easier to inspect because the original token and its stem can be compared directly.

#### Question 4 – Error Analysis of a Finite-State Morphological Parser

The following Python program is intended to identify plural nouns:

```
def parser(word):
 if word.endswith("s"):
 return word[:-2], "Plural Noun"
 else:
 return word, "Singular"

words = [
 "cars",
 "boxes",
 "cities",
 "children",
 "books"
]

for w in words:
 print(w, "->", parser(w))
```

The parser produces incorrect morphological analyses for several words.

#### Tasks

1. Identify all logical errors in the program.
2. Explain why the parser fails for different plural formation rules.
3. Modify the program to correctly handle:
  - Regular plurals
  - Words ending in -es
  - Words ending in -ies

- Irregular plurals

4. Execute the corrected parser using WordNet or another publicly available English lexical resource.
5. Compare the original and corrected outputs.
6. Identify the limitations of the rule-based finite-state approach.
7. Explain how the corrected parser improves morphological analysis.

1. Logical errors: The condition `word.endswith("s")` is too general. The expression `word[:-2]` removes two characters even for a regular plural such as `cars`, producing “car” only by coincidence. For `books` it produces “book”, but for `buses` it produces “bus” while the rule does not explicitly model `-es`. It cannot handle `-ies` forms such as `cities` and cannot handle irregular plurals such as `children`. It also does not verify whether the word is actually a plural noun.

2. Why the parser fails: English plural formation is not governed by one rule. Regular nouns normally add `-s`, nouns ending in `s`, `x`, `z`, `ch`, or `sh` often take `-es`, consonant+`y` changes to `-ies`, and some nouns have irregular forms such as `child` → `children`. Therefore, a single suffix test is insufficient.

3. Corrected program: ```python

```
def parser(word):
 irregular = {
 "children": "child",
 "men": "man",
 "women": "woman",
 "mice": "mouse",
 "geese": "goose"
 }

 if word in irregular:
 return irregular[word], "Plural Noun"

 if word.endswith("ies") and len(word) > 3:
 return word[:-3] + "y", "Plural Noun"

 if word.endswith(("ses", "xes", "zes", "ches", "shes")):
 return word[:-2], "Plural Noun"

 if word.endswith("s") and not word.endswith("ss"):
 return word[:-1], "Plural Noun"

 return word, "Singular"
```
```

4–5. Example corrected output: `cars` → `car`, Plural Noun

`boxes` → `box`, Plural Noun

`cities` → `city`, Plural Noun

`children` → `child`, Plural Noun

`books` → `book`, Plural Noun

Original output for `cars` and `books` happens to look correct, but `boxes` and `cities` are not handled by explicit rules, and `children` is incorrectly treated as singular. The corrected parser handles these cases explicitly.

6. Limitations: A rule-based finite-state approach is fast and interpretable, but it cannot reliably recognize every English noun without a large lexical exception list. It may also fail for ambiguous words, foreign plurals, spelling exceptions, and words that end in `s` but are not plural nouns. WordNet or another lexical resource can reduce these errors by supplying known lemma–word relationships.

7. Improvement: The corrected parser uses ordered morphological rules and an irregular-word lexicon. This improves segmentation and lemma identification while keeping the parser simple and efficient. It is more accurate than the original single-condition program for common English plural patterns.

Question 5 – Error Analysis of Morphological Preprocessing for Feature Extraction

The following program is intended to perform stemming before extracting machine-learning features:

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
    "connected connection connecting connectivity",
    "studies studied studying study",
    "organize organized organizer organization"
]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [
    stemmer.stem(word)
    for word in vectorizer.get_feature_names_out()
]
```

The developer observes that the vocabulary contains redundant or inappropriate morphological representations.

Tasks

1. Identify the preprocessing error in the program.
2. Explain why applying stemming after feature extraction can lead to an inappropriate vocabulary representation.
3. Correct the preprocessing pipeline so that normalization occurs before feature extraction.
4. Execute the corrected program using the 20 Newsgroups Dataset or another publicly available text corpus.
5. Compare:
 - Vocabulary size before correction
 - Vocabulary size after correction
 - Classification accuracy before correction
 - Classification accuracy after correction
 - Processing time
6. Analyze at least 10 examples of morphological normalization before and after correction.
7. Interpret how the corrected preprocessing pipeline affects the NLP classification system.

1. Preprocessing error: The program extracts the vocabulary using CountVectorizer first and only then applies Porter stemming to the feature names. Therefore, the vector matrix X still represents the original unnormalized words. The variable features contains stems, but those stems are not used to rebuild X.

2. Why this is inappropriate: Feature extraction creates the model vocabulary before stemming. Thus connected, connection, connecting, and connectivity may remain separate columns even though the developer intended morphological normalization. Stemming the feature names afterward does not merge the already-created columns, so the classifier still receives the redundant representation.

3. Correct pipeline: ```python
from nltk.stem import PorterStemmer

```

from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

def normalize(text):
    return " ".join(stemmer.stem(w.lower()) for w in text.split())

normalized_docs = [normalize(doc) for doc in documents]

vectorizer = CountVectorizer()
X = vectorizer.fit_transform(normalized_docs)

print(vectorizer.get_feature_names_out())
'''

```

4. Execution: On a real 20 Newsgroups experiment, the same normalization function should be applied to the training and test text before CountVectorizer is fitted. This prevents vocabulary leakage and ensures that the classifier sees the same representation during training and testing.

5. Comparison: For the three supplied documents, the unnormalized vocabulary contains separate forms such as connected, connection, connecting, connectivity, studies, studied, studying, study, organize, organized, organizer, and organization. After stemming, several forms are mapped to common stems, reducing the number of distinct vocabulary items. Exact classification accuracy and processing time depend on the chosen dataset, train/test split, hardware, and classifier; they should be measured by running the corrected experiment rather than inventing fixed values.

6. Ten normalization examples: 1. connected → connect

2. connection → connect

3. connecting → connect

4. studies → studi

5. studied → studi

6. studying → studi

7. study → studi

8. organized → organ

9. organizer → organ

10. organization → organ

These examples show both the benefit of vocabulary reduction and the risk of over-stemming, because Porter stems are not necessarily valid dictionary words.

7. Interpretation: Normalizing before feature extraction can reduce vocabulary size, sparsity, and memory usage. It may improve classification when different grammatical forms carry the same information. However, aggressive stemming can remove useful distinctions and sometimes reduce accuracy. Therefore, the preprocessing method should be evaluated on the target dataset using accuracy or F1-score together with processing time.

Answer Note: Dataset-specific accuracy, classification accuracy, and processing-time values are not supplied in the question paper. Where numerical examples are used, they are clearly identified as illustrative; actual corpus results should be obtained by executing the corrected programs on the selected dataset.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1	3	3	3	3	3	15	78
2	4	3	3	3	3	16	
3	3	3	3	4	3	16	
4	4	3	3	3	3	16	
5	3	3	3	3	3	15	

Faculty In-charge	Course Coordinator	Program Director
Dr. T. Kumaragurubaran		
Signature: Kumaragurubaran T	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____